EDSL Shared-State Semantics

Status: **Draft 0.1**

This directory specifies the meaning of the EDSL shared-state DSL independently of any particular local or remote implementation.

The normative words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** have their usual RFC 2119 meanings. Code examples are informative unless a section labels them normative.

How to read this specification

Each formal section is followed where useful by an informative worked example. The examples use one activity poll throughout:

```
activities = ["bike ride", "sailing", "hike", "beach day"]

fields = {
    "votes": state_field(
        T.map(T.text(), T.choice(activities)),
        initial={},
    ),
}

vote = Command(
    inputs={
        "voter": T.text(),
        "activity": T.choice(activities),
    },
    effects=(
        put("votes", input_("voter"), input_("activity"), once=True),
    ),
)
```

The poll is intentionally simple. It lets the reader connect the same concrete field, command, input, state, and view to the notation in later sections.

Artifacts

- `01-data-model.md` defines machines, states, scopes, and identifiers.
- `02-expressions-and-types.md` defines expression evaluation and static typing.
- `03-command-transitions.md` defines requirements, effects, and atomic commit.
- `04-concurrency-and-reads.md` defines per-scope ordering, retries, and reads.
- `05-views-and-capabilities.md` defines views and information access.
- `06-lifecycle.md` distinguishes complete, terminal, closed, and settled.
- `07-remote-and-security.md` defines local/remote equivalence and resource limits.
- `08-versioning.md` defines compatibility and registered capabilities.
- `machine.schema.json` defines the serialized envelope and structural grammar.
- `test-vectors/` contains implementation-independent examples.
- `run_vectors.py` executes the vectors against the reference interpreter.

Central semantic promise

For one scope, committed commands form a serial history:

```
S0 --c1--> S1 --c2--> S2 ... --cn--> Sn
```

Each command evaluates its requirement and every effect expression against one immutable committed input snapshot. It proposes one complete replacement state. The runtime validates and commits all of that state or none of it.

Reads name the exact snapshot and viewer context used to render a view. Local and remote implementations MUST produce the same observable result for the same versioned machine, history, context, and registered capabilities.

Notation

Expression evaluation is written:

$$M, S, I, X, L \vdash e \Downarrow v$$

where `M` is the machine, `S` state, `I` command inputs, `X` execution context, `L` lexical collection locals, `e` an expression, and `v` its value.

The environments have different ownership and lifetimes:

- S is the persistent, versioned machine-state snapshot. Commands may propose changes to its declared fields.
- I is the finite map of actual arguments supplied to this command invocation.
- X is read-only execution context supplied by EDSL, such as authenticated participant identity, group, interview ID, viewer role, run round, and lifecycle status. A machine cannot modify X.
- L contains temporary bindings introduced while evaluating collection expressions and disappears after evaluation.

A command transition is written:

$$M \vdash \langle S, c, I, X \rangle \longrightarrow \langle S', O \rangle$$

where `S'` is the committed or unchanged state and `O` is advisory outcome data.

Here c and I are intentionally separate. The command identifier c selects one definition from the machine's command map D; I supplies the actual input values for this invocation. For example:

$$c = \mathit{vote} \qquad I = \{\mathit{voter} \mapsto \text{Amina}, \mathit{activity} \mapsto \text{hike}\}$$

The same command $c = \mathit{vote}$ can be invoked many times with different input maps I.

S is not part of X because it has different semantics. S is durable shared data, has a snapshot ID, participates in concurrency control, and is the target of atomic transitions. X is immutable authority and execution metadata that may differ for two readers of the same S. Keeping them separate prevents a machine from modifying identity or authorization data and permits viewer-specific views over one shared snapshot.

Conformance

An implementation conforms to a DSL version only if it:

1. accepts every valid test vector for that version;
2. rejects every invalid vector with the specified error class;
3. produces the specified state, view, completion, and outcome values;
4. implements the security and resource requirements in this specification.

Passing the included reference vectors is necessary but not sufficient for production conformance.

1. Data model

Machine

A machine is a labeled record:

$$M = \left\langle \begin{array}{rcl} \mathit{name} & = & N, \\ \mathit{constants} & = & C, \\ \mathit{fields} & = & F, \\ \mathit{commands} & = & D, \\ \mathit{view} & = & V, \\ \mathit{complete} & = & P, \\ \mathit{close} & = & K, \\ \mathit{capabilities} & = & A \end{array} \right\rangle$$

The symbols name the following components:

- N is the diagnostic machine name and MUST NOT affect behavior.
- C is an immutable finite map of constants. Its values MUST be serializable DSL values.
- F is a finite map from field names to `(type, initial-expression)`.
- D is a finite map from command names to command definitions.
- V is a finite map from public result names to expressions.
- P is an optional Boolean completion predicate.
- K is an ordered finite sequence of close effects.
- A is the manifest of exact registered algorithm names and versions.

The labeled form is normative. Implementations MAY use a class, dictionary, or other representation, but field order MUST NOT affect machine semantics.

Worked example: the activity-poll machine

For the activity poll, the labeled machine components are approximately:

$$\begin{array}{rcl} N & = & \text{ActivityPoll}, \\ C & = & \{\mathit{activities} \mapsto [\text{bike ride}, \text{sailing}, \text{hike}, \text{beach day}]\}, \\ F & = & \{\mathit{votes} \mapsto (\text{Map}[\text{Text}, \text{Activity}], \{\})\}, \\ D & = & \{\mathit{vote} \mapsto \text{the vote command definition}\}, \\ V & = & \{\mathit{votes} \mapsto \text{field}(\mathit{votes}), \mathit{voteCount} \mapsto |\,\text{field}(\mathit{votes})|\}, \\ P & = & \text{null}, \\ K & = & [\,], \\ A & = & \{\} \end{array}$$

The poll has no completion predicate, close effects, or registered algorithms. Those empty components remain part of the machine shape, making serialization and validation uniform across simple and complex machines.

Values

DSL values are:

```
null | boolean | finite number | text | sequence[value] | map[value, value]
```

NaN and positive or negative infinity are not valid numbers. Implementations MUST enforce configured limits on nesting, string length, collection length, and serialized size.

State

A state is a finite map containing exactly the machine's declared fields. A state is valid when every field value recursively satisfies its declared type.

The initial state is obtained by evaluating field initial expressions in field declaration order. An initial expression MAY read constants and previously initialized fields. It MUST NOT read inputs, viewer context, or lexical locals.

For the poll:

$$S_0 = \{votes \mapsto \{\}\}$$

After Amina's first accepted vote, a later snapshot may be:

$$S_1 = \{votes \mapsto \{\text{Amina} \mapsto \text{hike}\}\}$$

SharedState and scope

A `SharedState` space is a finite map from machine names to machine instances:

$$Space = \text{Map}[MachineName, (MachineDefinition, MachineState)]$$

A `SharedStateMap` is a map from scope keys to spaces:

$$SharedStateMap = \text{Map}[ScopeKey, Space]$$

Selecting an absent scope MAY create a space from the map's declared initial definition. Scope creation MUST be atomic and deterministic.

Machines in different scopes have no implicit access to each other's state. Machines in one space also have no cross-machine access unless an explicit, typed capability is introduced by a future DSL version.

Snapshot and event identifiers

Every committed state version has an immutable snapshot ID. Every attempted command and explicit read has a globally unique event ID. Identifiers MUST NOT be reused, even when a command produces no state change.

The identifier representation is implementation-defined; equality and durable serialization are required.

2. Expressions and types

Environments

Expression evaluation receives five environments:

- constants `C`;
- immutable state snapshot `S`;
- validated command input `I`;
- capability-checked execution context `X`;
- lexical collection locals `L`.

A reference MUST resolve within its declared namespace. Missing references are validation errors, except for context references declaring an explicit default.

Type judgment

Static typing is written:

$$\Gamma \vdash e : \tau$$

The type environment is the disjoint union:

$$\Gamma = \Gamma_C \uplus \Gamma_F \uplus \Gamma_I \uplus \Gamma_X \uplus \Gamma_L$$

where the components contain types for constants, fields, command inputs, context capabilities, and lexical locals. Context entries additionally carry a disclosure label:

$$\Gamma_X(x) = (\tau, \ell)$$

The label ℓ is checked by the information-flow rules in Section 5 and does not change the ordinary value type τ.

Representative rules:

$$\frac{\Gamma_F(f) = \tau}{\Gamma \vdash \text{field}(f) : \tau}\ \text{T-Field}$$

Map lookup without a default returns an optional value:

$$\frac{\Gamma \vdash m : \text{Map}[K, V] \qquad \Gamma \vdash k : K}{\Gamma \vdash \text{get}(m, k) : \text{Optional}[V]}\ \text{T-Get-Optional}$$

Map lookup with a default of the map's value type returns a non-optional value:

$$\frac{\Gamma \vdash m : \text{Map}[K, V] \qquad \Gamma \vdash k : K \qquad \Gamma \vdash d : V}{\Gamma \vdash \text{get}(m, k, d) : V}\ \text{T-Get-Default}$$

Conditional branches are combined with the least defined type join:

$$\frac{\Gamma \vdash p : \text{Boolean} \qquad \Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2 \qquad \tau_1 \sqcup \tau_2 = \tau}{\Gamma \vdash \text{choose}(p, e_1, e_2) : \tau}\ \text{T-Choose}$$

The required joins are:

$$\tau \sqcup \tau = \tau$$

$$\tau \sqcup \text{Null} = \text{Null} \sqcup \tau = \text{Optional}[\tau]$$

$$\text{Optional}[\tau] \sqcup \tau = \tau \sqcup \text{Optional}[\tau] = \text{Optional}[\tau]$$

No implicit join exists for unrelated types. For example, $\text{Number} \sqcup \text{Text}$ is undefined unless a future DSL version introduces an explicitly declared union type. An undefined join is a creation-time type error; it does not silently widen to `Any`.

Worked example: constructing the poll's type environment

While checking the `vote` command, the relevant environments include:

$$\begin{aligned}
\Gamma_C(\textit{activities}) &= \text{Sequence}[\text{Activity}], \\
\Gamma_F(\textit{votes}) &= \text{Map}[\text{Text}, \text{Activity}], \\
\Gamma_I(\textit{voter}) &= \text{Text}, \\
\Gamma_I(\textit{activity}) &= \text{Activity}, \\
\Gamma_X(\textit{current.agent.name}) &= (\text{Text}, \text{Participant}), \\
\Gamma_L &= \emptyset.
\end{aligned}$$

Here `Activity` is the finite choice type containing the four configured activities. There are no lexical locals because the command does not contain a map, filter, or similar binding expression.

The expression:

```
field("votes").get(input_("voter"))
```

is typed in three steps:

$$\Gamma \vdash \text{field}(\textit{votes}) : \text{Map}[\text{Text}, \text{Activity}]$$

$$\Gamma \vdash \text{input}(\textit{voter}) : \text{Text}$$

and therefore:

$$\Gamma \vdash \text{get}(\text{field}(\textit{votes}), \text{input}(\textit{voter})) : \text{Optional}[\text{Activity}]$$

The optional result reflects that this voter may not have voted yet.

By contrast:

```
field("votes").get(input_("voter"), "hike")
```

has type `Activity`, because its default is also an `Activity`.

A context-dependent expression illustrates the disclosure component:

```
current.agent.name
```

has ordinary value type `Text`, while its label records that it identifies the current participant. Type checking answers "is this text?"; information-flow checking separately answers "may this view disclose it?"

An implementation MUST recursively validate sequence element types, map key and value types, and record fields. Top-level container checking is insufficient.

Evaluation

Pure expressions MUST NOT modify state or perform I/O. Evaluation is deterministic for identical environments and semantic versions.

Arithmetic MUST reject division by zero and non-finite results. Positional access MUST reject an invalid index unless guarded by a lazy expression.

Laziness

`choose`, Boolean `and`, and Boolean `or` are short-circuiting:

$$\frac{M, S, I, X, L \vdash p \Downarrow \text{true} \qquad M, S, I, X, L \vdash e_1 \Downarrow v}{M, S, I, X, L \vdash \text{choose}(p, e_1, e_2) \Downarrow v} \text{ E-Choose-True}$$

$$\frac{M, S, I, X, L \vdash p \Downarrow \text{false} \qquad M, S, I, X, L \vdash e_2 \Downarrow v}{M, S, I, X, L \vdash \text{choose}(p, e_1, e_2) \Downarrow v} \text{ E-Choose-False}$$

The absent branch premise is intentional: the unselected expression is not evaluated.

Short-circuit conjunction and disjunction are:

$$\frac{M, S, I, X, L \vdash p \Downarrow \text{false}}{M, S, I, X, L \vdash p \wedge q \Downarrow \text{false}} \text{ E-And-False}$$

$$\frac{M, S, I, X, L \vdash p \Downarrow \text{true} \qquad M, S, I, X, L \vdash q \Downarrow v}{M, S, I, X, L \vdash p \wedge q \Downarrow v} \text{ E-And-True}$$

The rules for $\vee$ are dual.

The unselected branch MUST NOT be evaluated. Similarly, the right operand of `and` is skipped after false and the right operand of `or` is skipped after true.

Collection operations

Map, filter, reduction, and sorting operate only on finite collections. Their lexical locals are visible only inside their own expression body.

Stable sorting MUST preserve input order among records equal under all declared sort keys. Reducers MUST define empty-collection behavior explicitly. It is an error to invoke a reducer for which the runtime lacks the exact semantic version.

Expression safety

The language MUST NOT provide arbitrary callbacks, imports, evaluation of code, reflection, unrestricted object attributes, filesystem or network access, general recursion, or user-defined unbounded loops.

3. Command transitions

Command definition

A command contains:

`Command = (inputs, optional requirement, ordered effects, timing)`

Inputs MUST match the declaration exactly and recursively satisfy their types.

Formally, c is a command identifier and $D(c)$ is its definition in machine M's command map. I is the map of actual values supplied to this particular invocation. Command lookup therefore precedes input validation:

$$d = D(c) \qquad \mathrm{dom}(I) = \mathrm{dom}(\mathrm{inputs}(d))$$

Normative execution algorithm

```
def execute(machine, committed_state, command_name, inputs, context):
    command = machine.commands[command_name]
    validate_inputs_exactly(command.inputs, inputs)
    validate_context_capabilities(command, context)

    snapshot = freeze(committed_state)

    if command.require is not None:
        if not evaluate(command.require, snapshot, inputs, context):
            return unchanged(snapshot, reason="requirement_not_met")

    evaluated = [
        evaluate_effect(effect, snapshot, inputs, context)
        for effect in command.effects
    ]

    proposed = copy(snapshot)
    for effect in evaluated:
        proposed = apply_effect(effect, proposed)

    validate_complete_state(machine.fields, proposed)
    enforce_resource_limits(proposed)
    return atomic_commit(proposed)
```

All effect expressions MUST evaluate against `snapshot`. Effects are applied in declaration order to `proposed`.

Transition judgments

Let evalEffects(E, S, I, X) evaluate all expressions in effect sequence E against immutable snapshot S. Let applyEffects$(S, \widehat{E})$ apply the resulting concrete effects in order.

A successful changing command is:

$$\frac{\text{validInputs}(M, c, I) \qquad M, S, I, X, \emptyset \vdash \text{require}(c) \Downarrow \text{true} \qquad \widehat{E} = \text{evalEffects}(\text{effects}(c), S, I, X) \qquad S' = \text{applyEffects}(S, \widehat{E})}{M \vdash \langle S, c, I, X \rangle \longrightarrow \langle S', \text{changed} \rangle}$$

An unmet requirement leaves state unchanged:

$$\frac{\text{validInputs}(M, c, I) \qquad M, S, I, X, \emptyset \vdash \text{require}(c) \Downarrow \text{false}}{M \vdash \langle S, c, I, X \rangle \longrightarrow \langle S, \text{requirement\_not\_met} \rangle} \text{ C-Require-False}$$

A valid command whose concrete effects make no change commits no new state contents, though it still receives a command-event ID:

$$\frac{\text{validInputs}(M, c, I) \qquad \text{require}(c) \Downarrow \text{true} \qquad \text{applyEffects}(S, \widehat{E}) = S}{M \vdash \langle S, c, I, X \rangle \longrightarrow \langle S, \text{unchanged} \rangle} \text{ C-No-Change}$$

If the proposed state is invalid, the transition fails and commits nothing:

$$\frac{S' = \text{applyEffects}(S, \widehat{E}) \qquad \neg\, \text{validState}(M, S')}{M \vdash \langle S, c, I, X \rangle \longrightarrow \langle S, \text{validation\_error} \rangle} \text{ C-Invalid-State}$$

Effects

`set(f,v)` replaces field `f` with `v`.

$$\text{apply}(\text{set}(f, v), P) = P[f \mapsto v]$$

`set_once(f,v)` sets `f` only when its current proposed value is null; otherwise it is a no-op.

$$\text{apply}(\text{setOnce}(f, v), P) = \begin{cases} P[f \mapsto v], & P(f) = \text{null} \\ P, & \text{otherwise} \end{cases}$$

`put(f,k,v)` assigns `f[k] = v`.

`put(f,k,v,once=true)` assigns only when `k` is absent from the current proposed map; otherwise it is a no-op.

$$\text{apply}(\text{putOnce}(f, k, v), P) = \begin{cases} P[f \mapsto P(f)[k \mapsto v]], & k \notin \text{dom}(P(f)) \\ P, & \text{otherwise} \end{cases}$$

`append(f,v)` appends `v` to sequence field `f`.

$$\text{apply}(\text{append}(f, v), P) = P[f \mapsto P(f) +\!\!+ [v]]$$

`when(p,e)` applies effect `e` only when predicate `p`, evaluated against the immutable input snapshot, is true.

$$\text{apply}(\text{when}(p, e), P) = \begin{cases} \text{apply}(e, P), & S, I, X \vdash p \Downarrow \text{true} \\ P, & S, I, X \vdash p \Downarrow \text{false} \end{cases}$$

A no-op `set_once` or `put-once` does not cancel sibling effects. If sibling effects must occur only when insertion is possible, the shared condition MUST be stated in the command requirement or on every dependent effect.

Worked example: executing `vote`

Let the current state be empty:

$$S_0 = \{votes \mapsto \{\}\}$$

The command identifier and invocation inputs are:

$$c = vote \qquad I_1 = \{voter \mapsto \text{Amina}, activity \mapsto \text{hike}\}$$

The command definition $D(c)$ says to evaluate the key and value expressions against S_0 and I_1, producing the concrete effect:

$$\text{putOnce}(votes, \text{Amina}, \text{hike})$$

Because `Amina` is absent, the transition is:

$$M \vdash \langle S_0, c, I_1, X\rangle \longrightarrow \langle\{votes \mapsto \{\text{Amina} \mapsto \text{hike}\}\}, \text{changed}\rangle$$

Now invoke the same command with different inputs:

$$I_2 = \{voter \mapsto \text{Amina}, activity \mapsto \text{sailing}\}$$

The `putOnce` key already exists, so:

$$M \vdash \langle S_1, c, I_2, X\rangle \longrightarrow \langle S_1, \text{unchanged}\rangle$$

This example also shows why c and I are separate: `vote` selects one command definition, while each invocation supplies a different input map.

Outcomes

Outcome data is advisory. It MAY report processed, changed, no-op, validation, and conflict information. It MUST NOT be represented as proof that the caller holds the latest state. Durable truth is the committed event and snapshot history.

Registered effects

A registered effect invokes an allowlisted algorithm capability. It receives only evaluated plain data and a copy of proposed state. Its complete output MUST pass the same type and resource validation before commit. Failure commits nothing.

4. Concurrency and reads

Per-scope serializability

Committed commands for one scope and machine MUST be equivalent to a total serial order. A command's input snapshot is the committed snapshot immediately preceding it in that order.

$$S_0 \xrightarrow{c_1} S_1 \xrightarrow{c_2} S_2 \cdots \xrightarrow{c_n} S_n$$

Different scopes MAY execute concurrently. This specification imposes no global ordering across scopes.

Conflicts and retries

An implementation MAY use locks, compare-and-swap, optimistic concurrency, or a single-writer service. These mechanisms are not observable semantics.

After a conflict, the runtime MAY retry only by reevaluating the entire command against the new committed snapshot, including its requirement, effect expressions, and registered algorithms. It MUST NOT reuse a proposed state computed from a stale snapshot.

If a proposal computed from S_i loses a commit race to c_j, retry is a new evaluation judgment over S_{i+1}:

$$\frac{M \vdash \langle S_i, c_j, I_j, X_j \rangle \longrightarrow \langle S_{i+1}, O_j \rangle \qquad \text{conflict}(c_i, S_i)}{\text{retry}(c_i) = M \vdash \langle S_{i+1}, c_i, I_i, X_i \rangle} \textsc{Retry-New-Snapshot}$$

Commands SHOULD carry an idempotency key. Repeating one accepted command event with the same key MUST NOT create a second logical command.

Read operation

An explicit read is:

$$\text{read}(q, m, \sigma, v, x)$$

where q is scope, m machine, σ snapshot selector, v view, and x viewer context.

It returns a rendered view plus a read event containing:

- read ID;
- scope key;
- machine name and version;
- exact snapshot ID;
- view name or version;
- permitted viewer-context identity;
- interview and survey position.

A read does not promise that its snapshot remains current after return.

View rendering is captured by:

$$M, S_\sigma, X \vdash V \Downarrow r$$

and the read result is:

$$(r, \text{ReadEvent}(id, q, m, \sigma, V, X))$$

Just-in-time reads

Survey state reads SHOULD occur as explicit steps immediately before the questions that need them. Implementations MUST NOT silently move a read earlier than its declared survey position.

Example timeline

```
A reads S0
B reads S0
A proposes vote(Amina, hike)
B proposes vote(Amina, sailing)
A commits: S0 -> S1
B retries against S1: put-once is a no-op
final state: {Amina: hike}
```

Both commands have event IDs. Only the first changes the state.

5. Views and capabilities

Views

A view is a pure function of a committed state snapshot and a permitted context:

$$V : State \times Context \longrightarrow RenderedValue$$

Rendering MUST NOT change state. The read event records the exact snapshot and context identity used.

Context capabilities

`current` references are capabilities, not unrestricted object traversal. Every path MUST be declared by the runtime with a type and disclosure class.

Recommended classes:

Class	Examples	Public view
public	group ID, round number	Allowed
participant	viewer name, role	Allowed when needed
participant-private	private value, signal	Only that participant
scope-private	unrevealed ballots	Explicit policy only
administrative	interview ID, event IDs	Not participant-visible
secret	credentials, store tokens	Never accessible

The validator MUST compute which capabilities every view expression may read. A public view MUST be rejected if it can read a capability outside its declared disclosure policy.

Stored data labels

Fields MAY declare disclosure labels. Derivations inherit the most restrictive label of their inputs unless a reviewed declassification rule applies.

Aggregation is not automatically safe declassification. Counts, timing, completion, errors, and collection sizes can reveal private facts.

Prompt rendering

Only the rendered view—not raw machine state—MAY be placed into a model prompt. Internal state, command inputs, event metadata, and store references MUST NOT be implicitly added to prompts.

Errors and advisory outcomes

Participant-visible errors and outcomes MUST NOT reveal private state used to evaluate a requirement. A generic `requirement_not_met` result is safer than reporting the hidden value that caused it.

Worked example: two views of one snapshot

Suppose one bargaining snapshot contains both private values:

$$S = \{price \mapsto 60, buyerValue \mapsto 100, sellerCost \mapsto 35\}$$

The buyer and seller read the same S with different contexts:

$$X_B(\mathit{viewer.role}) = \mathit{buyer} \qquad X_S(\mathit{viewer.role}) = \mathit{seller}$$

A permitted view can render:

$$V(S, X_B) = \{\mathit{price} \mapsto 60, \mathit{yourValue} \mapsto 100\}$$

$$V(S, X_S) = \{\mathit{price} \mapsto 60, \mathit{yourCost} \mapsto 35\}$$

The shared snapshot does not change. Only the read-only context and rendered result differ. This is the practical reason state S and context X are kept separate.

6. Lifecycle

The terms complete, terminal, closed, and settled are distinct.

Complete

`complete(S)` is a pure predicate indicating that the machine has enough data for its intended activity. Completion does not itself modify state or prohibit writes.

$$\text{complete}_M(S) = \text{eval}(P_M, S) \in \{\text{true}, \text{false}\}$$

Terminal

A terminal state prohibits ordinary commands except those explicitly declared as terminal-safe. Negotiation acceptance and walking away are common terminal events.

A machine MAY define terminal as identical to complete, but this MUST be explicit.

Closed

Close is an explicit, uniquely identified transition. It prevents further ordinary writes and evaluates close effects against one committed snapshot.

$$\frac{\widehat{K} = \text{evalEffects}(K_M, S, \emptyset, X) \qquad S' = \text{applyEffects}(S, \widehat{K}) \qquad \text{validState}(M, S')}{M \vdash \langle S, \text{close}(\mathit{id}), X\rangle \longrightarrow \langle S', \text{closed\_and\_settled}\rangle}\ \textsc{Close}$$

Close MUST be idempotent by event ID. Retrying the same close event cannot apply settlement twice. A second distinct close request SHOULD return the already closed snapshot without rerunning effects.

$$\frac{\text{closed}(S, \mathit{id})}{M \vdash \langle S, \text{close}(\mathit{id}), X\rangle \longrightarrow \langle S, \text{already\_closed}\rangle}\ \textsc{Close-Idempotent}$$

Settled

A machine is settled after all close effects and registered settlement algorithms have committed successfully. Closed but unsettled is a recoverable operational state only when an implementation cannot make close and settlement one transaction; it MUST NOT be exposed as successful closure.

State diagram

```
open --commands--> open
open --completion predicate true--> complete
open/complete --terminal command--> terminal
open/complete/terminal --close--> closed+settled
```

Schedules MAY use completion or terminal predicates to stop new interview turns. Only the close transition changes the closed state.

Views over lifecycle

The context capability `current.closed` is false before close and true only for the committed closed snapshot. It supports sealed views without giving recipes write authority over lifecycle metadata.

7. Remote execution and security

Observable equivalence

For the same versioned machine, initial state, ordered command events, contexts, and registered capability versions, local and remote runtimes MUST produce the same:

- committed states;
- views;
- completion and terminal values;
- close settlement;
- semantic outcome classifications.

Identifiers, timestamps, physical storage layout, and retry counts MAY differ.

Untrusted recipes

Serialized machines are untrusted data. Parsing MUST NOT execute expressions or registered algorithms.

Before execution, both submitting client and server MUST perform:

1. envelope and schema validation;
2. DSL and capability version resolution;
3. operator and reducer allowlisting;
4. reference resolution;
5. recursive type checking;
6. context and information-flow checking;
7. static complexity analysis;
8. literal and declared-limit validation.

The authoritative security decision occurs on the execution server.

Resource limits

A runtime MUST enforce configured maxima for:

- AST nodes and depth;
- literal and string bytes;
- state bytes;
- collection items and nesting;
- generated collection items;
- sort input size;
- evaluation steps;
- command and close wall time;
- event and rendered-view bytes;

- registered algorithm memory and time.

Nested collection operations require conservative cost estimation. A recipe whose provable upper bound exceeds policy MUST be rejected before execution or require a separately authorized capability.

Numeric behavior

NaN and infinity are invalid. Division by zero, overflow beyond the supported numeric domain, invalid indexing, and invalid empty reductions MUST produce a deterministic validation or evaluation error and commit no state.

Registered algorithm isolation

Registered algorithms are trusted implementation code and part of the trusted computing base. They MUST:

- be allowlisted by exact name and version;
- accept and return plain typed values;
- be deterministic;
- have no ambient filesystem, network, subprocess, environment, or credential access;
- run within stricter time and memory limits;
- validate their whole proposed output before commit;
- include conformance, invariant, and adversarial tests.

User-provided Python callbacks MUST NOT execute remotely as algorithms.

Results provenance

`Results.shared_state` SHOULD contain incoming and outgoing snapshot references, read events, write events, machine versions, and capability versions. Public serialization MUST redact private inputs, store credentials, and administrative context unless explicitly requested by an authorized reader.

8. Versioning and compatibility

Envelope

Every serialized definition MUST declare:

```
{
  "dsl_version": "0.1",
  "requires": {
    "operators": {},
    "reducers": {},
    "algorithms": {}
  },
  "limits": {},
  "machine": {}
}
```

The server MUST reject unsupported versions. It MUST NOT silently reinterpret a recipe under newer semantics.

Semantic versioning

Changing any of the following requires a new semantic version:

- effect no-op or ordering behavior;
- lazy versus eager evaluation;
- empty-reducer behavior;
- tie-breaking or sort stability;
- numeric coercion or precision;
- retry or close behavior;
- algorithm matching or settlement behavior;

- context disclosure rules.

Adding a backward-compatible optional envelope field does not necessarily change expression semantics.

Capability manifest

Every non-core operator, reducer, and algorithm SHOULD be listed with an exact integer semantic version. A runtime may derive core operator requirements from the AST, but explicit manifests improve inspection and admission control.

Recipes MUST NOT invoke an algorithm absent from their declared manifest.

The draft prototype's inner `machine.algorithms` list MAY contain legacy references such as `lmsr_trade@1`. The normative version is the outer manifest. A validator MUST normalize legacy references and reject any disagreement. A future schema version SHOULD remove the redundant inner list.

State migration

Changing field names, field types, initial values for existing scopes, or meaningful constants may require a state migration. A migration is a separate, versioned, audited transition; loading old state into a new machine definition without validation is prohibited.

Compatibility identity

A machine compatibility identity SHOULD hash the canonicalized DSL version, machine definition, capability manifest, and relevant limits. Results and state snapshots SHOULD record this identity.

Two definitions with different diagnostic names but identical canonical semantics MAY share a compatibility identity. Implementations must not depend on this optimization.